

Sistemas Distribuidos y Paralelos

Trabajo Entregable – Grupo 9

Graziani Martín 02117/8

Reyes Manuel 01852/0

1. Descripción del algoritmo secuencial.....	3
2. Estrategia y descripción de las etapas de diseño paralelo.....	3
• Descomposición o Particionado:.....	4
• Comunicación:.....	4
• Aglomeración:.....	5
• Mapeo:.....	5
3. Tiempos de ejecución, métricas y análisis de escalabilidad.....	6
• Tiempos de ejecución.....	6
• Speedup.....	6
• Eficiencia.....	7
• Balance de carga.....	8
• Análisis de escalabilidad.....	9
4. Uso de IA.....	10
Herramienta: ChatGPT.....	10
Herramienta: Gemini.....	11

1. Descripción del algoritmo secuencial

El programa resuelve el problema de las N-Reinas, que consiste en ubicar N reinas sobre un tablero de ajedrez de tamaño $N \times N$ de manera que ninguna de ellas pueda atacar a otra. Esto implica que no pueden compartir fila, columna ni diagonal.

El algoritmo secuencial implementa una búsqueda recursiva y aprovecha las simetrías del tablero para reducir el tamaño de búsqueda.

El valor de cada fila es un entero usado como máscara de bits, el bit encendido (valor alto) indica en qué columna está ubicada la reina. De esta forma se asegura que solamente haya una reina por fila y no es necesario almacenar el tablero completo. El uso de máscaras de bits permite realizar las validaciones de posiciones mediante operaciones binarias, considerablemente más rápidas y eficientes que recorrer estructuras de datos convencionales.

Para determinar si un tablero es válido se utiliza un backtracking recursivo. La idea es colocar una reina en una fila y verificar qué posiciones son válidas para la siguiente fila, para luego verificar de manera recursiva. Si en alguna fila no hay una posición válida, se retrocede (backtrack).

Además del backtracking, el algoritmo aprovecha las simetrías del tablero para evitar trabajo redundante. Una solución puede generar otras equivalentes mediante rotaciones o reflexiones, por lo que no es necesario explorarlas de forma independiente.

Durante la búsqueda se considera únicamente un subconjunto representativo del espacio de soluciones y luego se identifican las equivalencias por simetría. Esto permite distinguir entre soluciones únicas y soluciones totales, reduciendo significativamente el tiempo de cómputo y mejorando el rendimiento del algoritmo.

2. Estrategia y descripción de las etapas de diseño paralelo

El algoritmo secuencial de N-Reinas fue paralelizado utilizando una arquitectura híbrida basada en MPI y pthreads, con dos procesos MPI que representan a los dos nodos utilizados del cluster proporcionado por la cátedra.

La estrategia adoptada consiste en dividir el espacio de búsqueda en múltiples tareas independientes obtenidas a partir de las primeras decisiones de ubicación de las reinas. Estas tareas son distribuidas entre los procesos MPI, mientras que cada proceso utiliza pthreads para explorar en paralelo las distintas ramas del árbol de búsqueda mediante backtracking. De esta forma se combina paralelismo entre nodos y paralelismo compartido dentro de cada nodo.

Según Foster y Grama, el diseño de la paralelización puede describirse siguiendo las siguientes etapas:

1. Descomposición o Particionado
2. Comunicación
3. Aglomeración
4. Mapeo

- **Descomposición o Particionado:**

La etapa de particionado consiste en descomponer el problema original en tareas independientes.

Dado que el problema de N-Reinas se resuelve mediante una búsqueda sobre un árbol de exploración, donde cada nodo representa una configuración parcial del tablero, se utilizan distintas ramas de dicho árbol como unidades de trabajo.

En el programa implementado, cada tarea representa un subárbol de búsqueda generado a partir de colocar una reina en una posición inicial. La tarea no resuelve solamente una posición aislada, sino que continúa la exploración recursiva desde dicha configuración hasta encontrar todas las soluciones. De esta manera, cada worker toma una tarea pendiente, inicializa el estado correspondiente del tablero y ejecuta el backtracking sobre el subárbol asignado.

Concretamente, una tarea consiste en:

- definir una posición inicial válida,
- explorar el subárbol recursivamente,
- llevar una cuenta de las posibles soluciones.

Dentro de cada proceso, los hilos comparten una estructura de tareas pendientes desde la cual obtienen nuevos trabajos a medida que finalizan los anteriores. Para garantizar acceso seguro a esta estructura se utiliza exclusión mutua durante la asignación de tareas.

Este paralelismo es entonces un paralelismo de datos, ya que cada tarea realiza el mismo trabajo pero sobre distintos datos. La granularidad obtenida es más fina que una división de subárboles completos, lo que permite distribuir la carga de la manera más uniforme posible entre las distintas unidades de procesamiento.

Esta descomposición es de tipo exploratoria, dado que el algoritmo recorre un árbol de búsqueda y sus ramas representan distintas configuraciones parciales del tablero. Durante la exploración se generan nuevas alternativas y se descartan aquellas en las que se detectan que no llevan a soluciones del problema, reduciendo así el espacio de búsqueda.

- **Comunicación:**

La comunicación entre procesos se limita a dos momentos. Inicialmente se distribuyen las tareas entre los procesos MPI para que cada uno explore una parte del espacio de búsqueda. Finalizada la exploración, los resultados parciales obtenidos por

cada proceso se combinan para calcular la cantidad total de soluciones.

Durante la etapa de cómputo no es necesario realizar comunicaciones adicionales entre procesos, ya que las tareas son independientes entre sí. Esto reduce significativamente el overhead de comunicación y favorece la escalabilidad de la solución.

Dentro de cada proceso, los hilos únicamente comparten el pool de tareas pendientes, protegida por el mutex, y los contadores de resultados, manteniendo un nivel de comunicación y sincronización reducido.

- **Aglomeración:**

La descomposición elegida genera tareas pequeñas de granularidad relativamente gruesa, cada una de ellas representa a un subárbol completo de búsqueda.

Debido a esta granularidad, no se aplica una etapa adicional de aglomeración. Esto genera un desbalance de cargas debido a la variedad de tamaños de los subárboles asociados a cada tarea.

Una potencial mejora a la solución sería reducir la granularidad de las tareas, subdividiendo el espacio de búsqueda en tareas más chicas. Esto permitiría una distribución más uniforme de tareas en las unidades de trabajo, aumentando la eficiencia y escalabilidad del algoritmo.

- **Mapeo:**

La estrategia de mapeo utilizada es híbrida. En una primera instancia, las tareas generadas se distribuyen entre los procesos MPI de manera estática. Como las distintas tareas pueden requerir cantidades de cómputo diferentes, se busca reducir lo más posible el desbalance de carga inicial entre los procesos.

Dentro de cada proceso MPI, el mapeo entre tareas y threads se realiza dinámicamente mediante un pool de tareas compartido. Los hilos obtienen nuevas tareas a medida que finalizan las anteriores, permitiendo que aquellos que terminan primero continúen procesando trabajo pendiente en lugar de permanecer inactivos.

Esta estrategia combina un reparto inicial de tareas entre procesos MPI con una asignación dinámica entre hilos dentro de cada proceso. El objetivo es reducir los costos de comunicación entre nodos y, al mismo tiempo, mejorar el aprovechamiento de los recursos de cómputo disponibles, buscando mitigar el desbalance de cargas de la exploración del espacio de búsqueda.

3. Tiempos de ejecución, métricas y análisis de escalabilidad

Los resultados obtenidos para las distintas dimensiones del tablero son los siguientes:

- **Con 14 Reinas:** Número de resultados: 365.596
- **Con 15 Reinas:** Número de resultados: 2.279.184
- **Con 16 Reinas:** Número de resultados: 14.772.512
- **Con 17 Reinas:** Número de resultados: 95.815.104
- **Con 18 Reinas:** Número de resultados: 666.090.624

- **Tiempos de ejecución**

Se miden los tiempos totales de cómputo en las distintas configuraciones del cluster, y se toman los tiempos de cada hilo para calcular luego el speedup, la eficiencia y el balance de carga.

Tiempo (seg)	Carga de trabajo (N)				
UP	14	15	16	17	18
1	0,181224	1,113146	7,252204	50,174202	364,633939
4	0,055591	0,319865	2,260040	16,169393	120,689946
8	0,054513	0,301307	1,821964	11,947337	83,584579
16	0,053355	0,300531	1,823060	11,949947	81,816010

- **Speedup**

$$S = \frac{T_a}{T_b}$$

- T_a : Tiempo antes de la mejora.
- T_b : Tiempo después de la mejora.

Para calcular el speedup se utiliza como tiempo antes de la mejora el tiempo del proceso secuencial ($N=1$, $T=1$), o de un solo worker, y como tiempo después de la mejora los tiempos con más de un worker, respectivamente.

Speedup	Carga de trabajo (N)				
UP	14	15	16	17	18
1	1,00	1,00	1,00	1,00	1,00
4	3,26	3,48	3,21	3,10	3,02
8	3,32	3,69	3,98	4,20	4,36
16	3,40	3,70	3,98	4,20	4,46

Se puede observar que al aumentar la cantidad de unidades de procesamiento, el rendimiento mejora respecto de la versión secuencial. A medida que se incrementa el número de reinas, pasar de 4 a 8 UP produce un incremento significativo del speedup. Sin embargo, al pasar de 8 a 16, la mejora se estanca bastante para cualquier número de reinas.

Esto muestra que agregar más unidades de procesamiento no siempre produce una mejora proporcional, ya que aparecen factores asociados al desbalance de carga, la sincronización, la comunicación y la administración del trabajo.

- **Eficiencia**

$$E(P) = \frac{S(P)}{P}$$

- ***S***: *Speedup* utilizando *P* unidades de procesamiento.
- ***P***: Unidades de procesamiento utilizadas.

Para el cálculo de la eficiencia se utilizan los speedups escritos anteriormente, calculados con respecto al tiempo secuencial y se los divide por el número de workers o unidades de procesamiento.

Eficiencia (%)	Carga de trabajo (N)				
UP	14	15	16	17	18
1	100,00	100,00	100,00	100,00	100,00
4	81,50	87,00	80,22	77,58	75,53
8	41,56	46,18	49,76	52,50	54,53
16	21,23	23,15	24,86	26,24	27,85

A partir de los resultados obtenidos, se observa que la eficiencia disminuye a medida que aumenta la cantidad de unidades de trabajo. Esto se debe a que el speedup logrado no crece proporcionalmente a la cantidad de hilos utilizados.

A su vez, para un mismo número de UP, la eficiencia mejora levemente al aumentar N, ya que el costo de paralelización se reparte entre más trabajadores.

- **Balance de carga**

$$B = \frac{\text{Promedio}(T)}{\text{Máximo}(T)}$$

- **Promedio(T):** Promedio de los tiempos de ejecución de los hilos o procesos.
- **Máximo(T):** Tiempo de ejecución del hilo o proceso que más tardó.

Para calcular los balances de carga, se tomaron los tiempos de cada hilo, de éstos se elige el de mayor duración, y también se calcula el tiempo promedio de ejecución de cada configuración.

Balance de carga (%)	Carga de trabajo (N)				
UP	14	15	16	17	18
1	100,00	100,00	100,00	100,00	100,00
4	84,21	90,09	82,99	80,05	77,83
8	43,30	47,94	51,46	54,15	56,16
16	21,96	24,00	25,74	27,08	28,69

Los resultados muestran que el balance de carga disminuye al aumentar la cantidad de unidades de procesamiento, lo que indica una distribución cada vez menos uniforme del trabajo entre procesos.

Para una misma carga de trabajo, el desbalance se vuelve más evidente al escalar a más UP.

En cambio, al aumentar N, se observa una leve mejora del balance dentro de cada configuración, ya que el mayor tamaño del problema permite distribuir mejor el trabajo entre los procesos, a excepción del caso de 4 workers, donde el balance de carga disminuye al superar las 15 reinas.

● **Análisis de escalabilidad**

- **Ley de Amdahl:** En los resultados obtenidos se observa que, para todos los tamaños de tablero, el speedup aumenta al pasar de 1 a 4 y de 4 a 8 workers. Sin embargo, al incrementar la cantidad de workers de 8 a 16, la mejora se estanca en la mayoría de los casos. Este comportamiento es consistente con la Ley de Amdahl, ya que al aumentar el grado de paralelismo empieza a dominar la fracción no paralelizable del programa, junto con los costos adicionales de comunicación y distribución de tareas.
- **Ley de Gustafson:** La Ley de Gustafson analiza el comportamiento del paralelismo cuando aumenta el tamaño del problema, considerando que en problemas más grandes la proporción de trabajo paralelizable tiende a ser mayor. Se puede observar una tendencia relacionada analizando cómo varía el speedup al incrementar el tamaño del tablero para una cantidad fija de workers.
Para 4 workers no se observa un comportamiento compatible con la Ley de Gustafson, ya que el speedup no aumenta al crecer el tamaño del problema. En cambio, para 8 y 16 workers sí se observa una tendencia más compatible: al aumentar N, el speedup también crece. Esto indica que, para cargas de trabajo mayores, el algoritmo logra compensar mejor el overhead

de paralelización aunque el crecimiento sigue limitado por el desbalance de carga y por las secciones no paralelizables del programa.

- **Escalabilidad débil:** Para analizar la escalabilidad débil se estudia la tabla de eficiencia, específicamente las diagonales, donde se incrementan simultáneamente la carga de trabajo y el número de unidades de procesamiento.
Se verifica que en las diagonales, la eficiencia decae significativamente. Por ende, el algoritmo no presenta una buena escalabilidad débil. Esto se debe al desbalance de cargas, ya que algunas tareas corresponden a subárboles mucho más grandes que otros, provocando que ciertas unidades de procesamiento permanezcan ociosas mientras otras continúan trabajando.
- **Escalabilidad fuerte:** para estudiar la escalabilidad fuerte se analizan las columnas de la tabla de eficiencia, manteniendo fijo el tamaño del problema y aumentando la cantidad de unidades de procesamiento.
En los resultados se observa que, para cada tamaño de tablero, la eficiencia disminuye al incrementar el número de workers. Esto indica que el algoritmo posee una baja escalabilidad fuerte. Esta pérdida de eficiencia puede explicarse por el desbalance de carga generado por la diferencia de tamaño y costo computacional entre las distintas tareas, junto con el overhead generado por comunicar mayores cantidades de workers.

4. Uso de IA

Herramienta: ChatGPT

Prompt	Éxito	Observación
Cómo funcionan los backtracking en N-Reinas?	Parcial	Simplifica bastante el problema, pero lo explica bien.
Cómo funcionan las simetrías?	Sí	Explica todas las rotaciones y reflexiones posibles.
Generar estructura de pthreads + MPI	Sí	Da un esqueleto básico de un programa híbrido
Scripts de bash para el cluster y recaudos que se debe tener al usarlo	Sí	Da una estructura básica de script SLURM y explica bien los recaudos para no saturar el cluster y cómo ver los resultados correctamente
Diferencias MPI_Scatterv y MPI_Scatter y cuando usar cada uno	Sí	Explica perfectamente cómo usar, sus parámetros y en qué contexto es mejor cada uno, las limitaciones del MPI_Scatter y la complejidad extra de MPI_Scatterv, y brinda ejemplos para entender mejor las diferencias
Estrategias para el análisis de Foster para un problema híbrido MPI+Pthreads	Parcial	Simplifica bastante el análisis pero da ejemplos interesantes, formas de comunicación, tipos de granularidad y formas de mapeo
Diferencia entre resultados al ejecutar en cluster o local	Sí	Tras enviarle las prestaciones del cluster, explica las diferencias con la computadora local, justificando así los tiempos de ejecución.
Problema de tareas sin asignar al usar	Parcial	Explica que el número de tareas debe ser divisible por

MPI_Scatter		la cantidad de procesos y si esto no pasa, se debe usar MPI_Scatterv o asignar tareas manualmente. No dice nada de hacer padding.
Desbalance de cargas con distintas configuraciones	Sí	Justifica que en N-Reinas, no todas las ramas del árbol de búsqueda cuestan lo mismo. Algunas posiciones iniciales generan muchísimas combinaciones posibles y tardan mucho más; otras se podan rápido por simetrías o conflictos.

Herramienta: Gemini

Prompt	Éxito	Observación
Cómo funcionan los backtracking en N-Reinas?	Sí	Explica el algoritmo fila a fila en el tablero de manera muy clara
Cómo funcionan las simetrías?	Sí	Explica bien las rotaciones y reflexiones y agrega algo de código para ejemplificar
Generar estructura de pthreads + MPI	Sí	Da una estructura base del código y explica como ejecutarlo
Scripts de bash para el cluster y recaudos que se debe tener al usarlo	Sí	Da reglas básicas de cómo hacer los scripts y algunas listas de buenas prácticas de programación
Diferencias MPI_Scatterv y MPI_Scatter y cuando usar cada uno	Sí	Compara las sintaxis de cada uno, explica los offsets necesarios para implementar MPI_Scatterv y detalla cuándo conviene usar cada uno
Estrategias para el análisis de Foster	Sí	Explica detalladamente cada etapa, qué es importante en cada una y qué recaudos tomar para no generar overhead innecesario
Diferencia entre resultados al ejecutar en cluster o local	Sí	Explica los tiempos de reloj de procesadores actual (cuando se corre local) y los del cluster para justificar los distintos tiempos
Problema de tareas sin asignar al usar MPI_Scatter	Sí	Explica cómo funciona MPI_Scatter y que no tiene la flexibilidad para saber qué hacer con las tareas sobrantes tras la división. Propone usar padding o directamente MPI_Scatterv
Desbalance de cargas con distintas configuraciones	No	Entendió mal la consulta y brindó alternativas incorrectas para cambiar el código. No responde el problema planteado